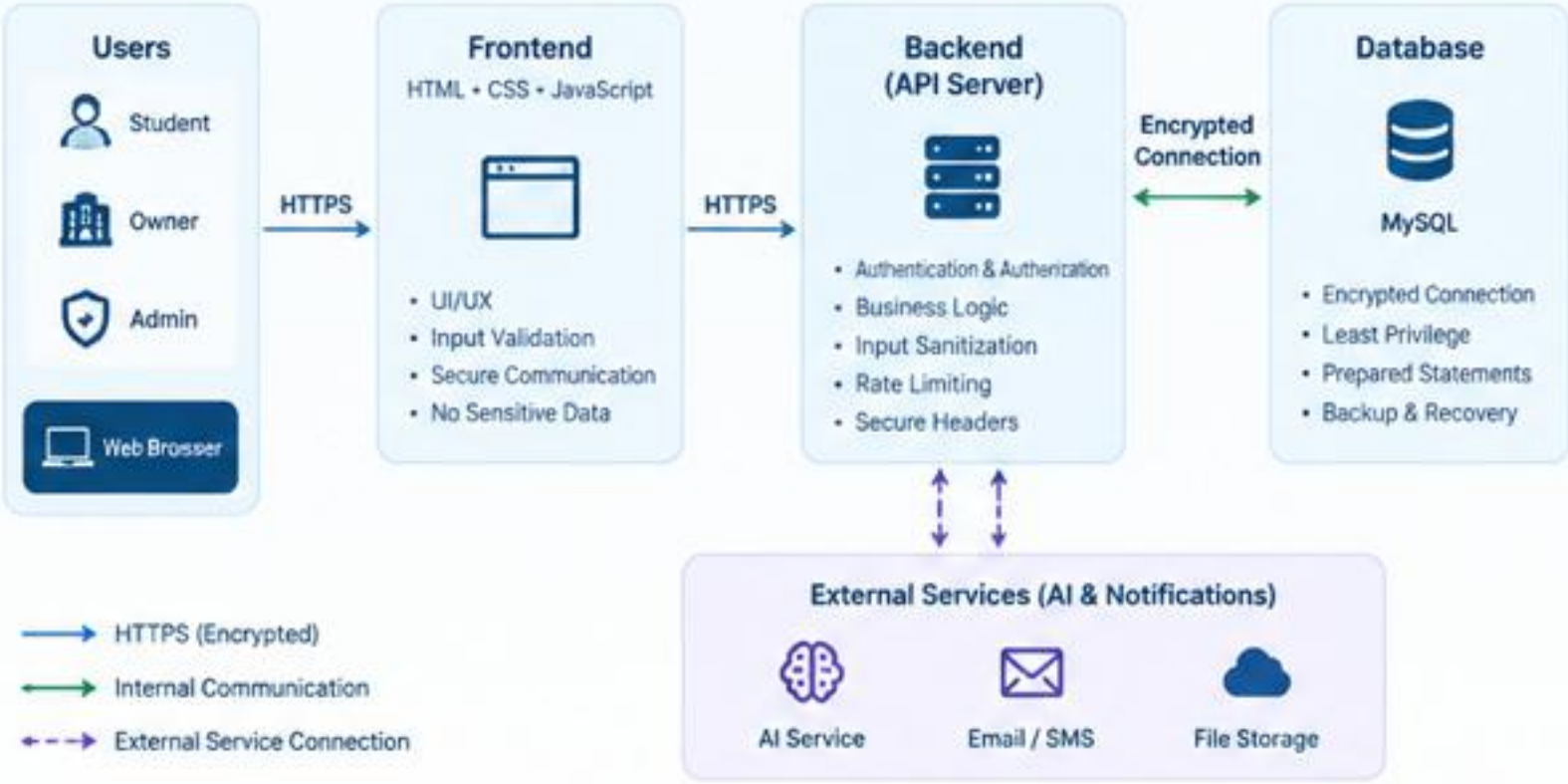
Campus Life Companion

Security Overview & Architecture

Safe Campus
Better Experience

1. System Architecture & Security Layers



2. Key Security Measures

Authentication & Authorization

- Secure Login & Logout
- Password Hashing (bcrypt)
- JWT / Secure Sessions
- Role-Based Access Control (RBAC)

Data Protection

- Input Validation & Sanitization
- Prevent SQL Injection
- Encrypt Sensitive Data
- Secure Cookies (HttpOnly, Secure)

API Security

- Token-Based Authentication
- Rate Limiting
- CORS Configuration
- Request Validation

Database Security

- Encrypted Connection (SSL)
- Least Privilege Access
- Prepared Statements
- Regular Backups

Web Application Security

- XSS Protection
- CSRF Protection
- Clickjacking Protection
- Secure Headers (CSP, HSTS)

Monitoring & Incident Response

- Logs & Monitoring
- Error Handling (No Information Leak)
- Backup Plan
- Vulnerability Scanning

Goal

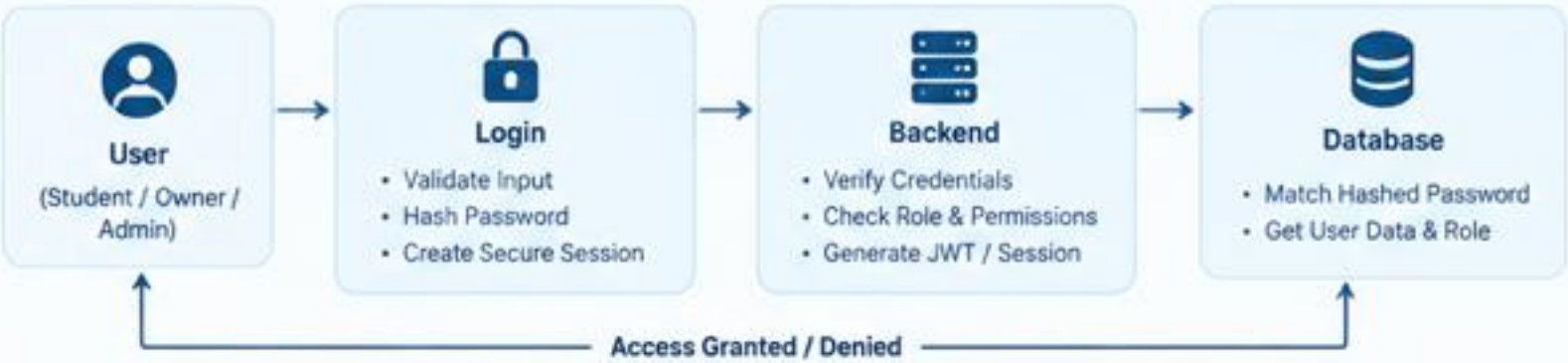
Build a secure, reliable, and user-friendly platform that protects user data, prevents common web attacks, and ensures safe interaction across all features.

Campus Life Companion

Security Implementation Plan

Secure Infrastructure
Protected Data
Trusted Community

3. Security Flow (Login & Access Control)



4. Security for Each Feature

Housing

- Verify User Identity
- Prevent IDOR
- Validate Uploaded Files
- Protect Location Data

Carpooling

- Validate Ride Requests
- Prevent Fake Listings
- User Reputation System
- Secure Location Handling

Reviews & Ratings

- Prevent Fake Reviews
- Validate User Permissions
- Sanitize Input
- Rate Limiting

Reports & Complaints

- Verify Reporter Identity
- Prevent Abuse & Spam
- Secure File Uploads
- Admin Access Control

Chat

- End-to-End Encryption (optional)
- Prevent XSS
- Validate Messages
- Block Malicious Content

AI Features

- Sanitize Input to AI
- Limit Usage & Rate Limit
- Protect API Keys
- Handle Sensitive Data

5. Tools & Technologies

Visual Studio Code	GitHub	Burp Suite	OWASP ZAP	Postman	MySQL	More Tools
Development	Version Control	Security Testing	Vulnerability Scanner	API Testing	Database	(Optional)

6. Security Responsibilities & Success Criteria

As Security Lead

- ✓ Define security requirements
- ✓ Review code and configurations
- ✓ Conduct vulnerability testing
- ✓ Coordinate with the development team
- ✓ Maintain security documentation

Successs Criteria

- ✓ No critical vulnerabilities
- ✓ Proper authentication & authorization
- ✓ Secure data handling
- ✓ All features tested and protected
- ✓ Secure deployment configuration